Git ハンズオン Day 1：書いて、記録して、公開する

今日のゴール

テキストファイルで自己紹介ドキュメントを書きながら、Gitの基本操作を身につけます。
最終的に、自分のGitHubリポジトリに公開するところまでやります。

1. GitHubアカウントの作成と認証設定（50分）

1-1. GitHubにサインアップする

1. ブラウザで https://github.com にアクセスします
2. 右上の「Sign up」をクリックします
3. 以下の情報を入力します

Email：

会社のメールアドレスを使ってください。

Password：

自分で決めたパスワードを入力します。パスワードの決め方は、この後のコラムを読んでください。

Username：

GitHubのユーザー名は、今後の仕事で長く使うものです。
チームの先輩やプロジェクトメンバーがあなたのユーザー名を目にすることになります。
以下の点を考えて決めてください。

- **誰だかわかる名前にする** — あなたの名前やイニシャルを含めると、チームで識別しやすくなります
  - 例： `taro-yamada` 、 `t-yamada` 、 `yamada-t`
- **業務で使って恥ずかしくない名前にする** — プライベートのニックネームやふざけた名前は避けましょう
- 後から変更もできますが、変更すると過去のURLが変わるため、最初にきちんと決めるのがおすすめです

4. 画面の指示に従って、メール認証を完了します

> **困ったら：** 認証メールが届かない場合は、迷惑メールフォルダを確認してください。

コラム：パスワードの取り扱いについて

GitHubに限らず、クラウドサービスのパスワードは**あなたの仕事そのものを守る鍵**です。
社会人として、パスワードの管理は最も基本的なセキュリティスキルのひとつです。

ここでしっかり身につけてください。

やってはいけないこと：

- パスワードを付箋に書いてモニターに貼る
- パスワードをメールやチャットで送る
- 他のサービスと同じパスワードを使い回す
- 名前や誕生日など、推測されやすい文字列を使う

パスワードの作り方のコツ：

- 15文字以上の長いパスワードを作りましょう
- 英大文字・小文字・数字・記号を組み合わせましょう
- 自分だけがわかるフレーズを元にすると、長くても覚えやすくなります
  - 例：好きな歌の歌詞の頭文字を取る、好きな文章をローマ字にして記号を混ぜる、など

「メモしておいて」と言われたら？

パスワードをメモすること自体が悪いわけではありません。
問題なのは、**メモの管理が甘くなること**です。
メモを取ると安心してしまい、デスクの上に放置したり、誰でも見られる場所に置いてしまいがちです。

もしメモを取る場合は：

- 他の人が見てもパスワードだとわからない形にする
- 物理的なメモは鍵のかかる場所に保管する
- 可能であれば、パスワードマネージャー（1Password、Bitwardenなど）の利用を検討する

> **覚えておいてください：**
> パスワードは「忘れても再発行できる」ものです。
> この後のハンズオンで、実際にパスワードの再発行を体験します。
> 大切なのは「忘れないこと」よりも「漏らさないこと」です。

1-2. 二要素認証（2FA）を設定する

GitHubでは、アカウントを守るために **二要素認証（2FA）** の設定が必要です。

二要素認証とは、パスワードに加えて**もうひとつの確認手段**を使ってログインする仕組みです。
たとえパスワードが漏れても、もうひとつの確認がなければログインできないため、安全性が大きく上がります。

> **身近な例：**
> 銀行のATMでは「キャッシュカード」と「暗証番号」の2つが必要ですよね。
> 二要素認証もこれと同じ考え方です。

今回は **Passkey（パスキー）** という方法を使って2FAを設定します。
みなさんに配布されたWindows PCだけで完結するので、スマートフォンは不要です。

Windows HelloのPINを確認する

PasskeyはWindows Helloの機能を利用します。
まず、Windows HelloのPINが設定済みか確認してください。

1. Windowsの「設定」を開きます（スタートメニュー → 歯車アイコン）
2. 「アカウント」→「サインインオプション」を開きます
3. 「PIN（Windows Hello）」が設定済みになっていることを確認します

> **PINが未設定の場合：** 講師の指示に従って、PINを設定してください。

GitHubでPasskeyを登録する

1. GitHubにログインした状態で、右上のアイコン → 「Settings」を開きます
2. 左メニューの「Password and authentication」をクリックします
3. 「Two-factor authentication」のセクションで2FAを有効にします
4. 設定方法の選択で「Passkey」または「Security key」を探します
5. 「Add a passkey」をクリックします
6. ブラウザがWindows Helloの認証を求めてきます
7. **PINを入力する**、または**指紋認証をする**と、Passkeyが登録されます
8. **リカバリーコード**が表示されます — これは必ず安全な場所に保存してください

> **リカバリーコードとは：**
> PCを紛失・初期化した場合に、アカウントにログインするための緊急用コードです。
> 印刷するか、安全な場所にメモして、大切に保管してください。
> このコードがないと、Passkeyが使えなくなった際にアカウントに入れなくなります。

> **困ったら：** この手順は画面が多いので、講師のスクリーンを見ながら一緒に進めましょう。

コラム：2FAの方法はひとつではない

今回はPasskeyを使いましたが、二要素認証にはいくつかの方法があります。
今後、業務で他のクラウドサービスを使うときにも出てくるので、覚えておきましょう。

方法	仕組み	特徴
Passkey （今回使った方法）	PCやスマホの生体認証・PINで本人確認する	アプリ不要。 設定したデバイスでのみ使える
認証アプリ （Authenticatorアプリ）	スマホアプリが30秒ごとに6桁の数字を生成する	業界で最も広く使われている方法。 スマホが必要
ブラウザ拡張	PCのブラウザ拡張が6桁の数字を生成する	スマホ不要だが、 PC変更時にデータの移行が必要

業界では **認証アプリ（Authenticatorアプリ）** が最も一般的です。
代表的なアプリは以下の2つです。

- **Microsoft Authenticator**
- **Google Authenticator**

どちらもiPhone（App Store）、Android（Google Play）で無料で入手できます。

コラム：配属後にAuthenticatorアプリへ移行するには

今回はPasskeyで2FAを設定しましたが、配属後にスマートフォンの認証アプリへ切り替えることをおすすめします。

Passkeyは登録したPCでしか使えないため、PCの故障・交換・初期化があると
Passkeyが使えなくなります。スマートフォンのAuthenticatorアプリなら、
PCが変わっても影響を受けません。

切り替えの手順（概要）

GitHubでは、**Passkeyを削除しなくても、認証アプリを追加できます。**
複数の2FA方法を並行して登録できるので、安全に移行できます。

1. GitHubにログイン → Settings → 「Password and authentication」を開く
2. 「Two-factor methods」のセクションで「Authenticator app」を追加する
3. 画面に表示されるQRコードを、スマホのAuthenticatorアプリで読み取る
4. アプリに表示される6桁の数字をGitHubに入力して登録完了
5. 認証アプリでログインできることを確認してから、必要に応じてPasskeyを削除する

> **ポイント：**
> 「古い方法を削除してから新しい方法を登録」ではなく、
> 「新しい方法を追加してから古い方法を削除」が鉄則です。
> 切り替え中にログインできなくなることを防げます。
>
> 万が一のために、リカバリーコードも手元にある状態で作業しましょう。

1-3. パスワード再発行を体験する

ここで一度GitHubから**ログアウト**して、「パスワードを忘れた」場合の再発行を体験します。

パスワードの再発行は、クラウドサービスを使っていれば必ず経験することです。
焦らず対応できるように、今のうちに練習しておきましょう。

ログアウトする

1. GitHubの右上のアイコンをクリック
2. 「Sign out」をクリック

パスワード再発行の手順

1. https://github.com のログイン画面を開きます

2. 「Forgot password?」をクリックします
3. 会社のメールアドレスを入力して「Send password reset email」をクリックします
4. メールを確認します（届くまで少し時間がかかることがあります）
5. メール内のリンクをクリックします
6. 新しいパスワードを設定します（先ほどのコラムを参考に、しっかりしたパスワードを作りましょう）
7. 新しいパスワードでログインします
8. Passkeyの認証を求められます — Windows HelloのPINを入力してください

> **ポイント：**
> パスワードを再発行した後のログインでも、2FAの確認（Passkey）が求められます。
> これが「二要素認証」の仕組みです。パスワードだけではログインできません。

ログインできましたか？ これで、パスワードを忘れても自分で対処できるようになりました。

1-4. Git Bashを開く

Windowsのスタートメニューから「Git Bash」を探して開きます。
黒い画面（ターミナル）が表示されればOKです。

1-5. Gitに自分の名前とメールアドレスを設定する

Git Bashに以下のコマンドを **1行ずつ** 入力して、Enterキーを押します。

```
git config --global user.name "自分の名前"
```

```
git config --global user.email "会社のメールアドレス"
```

> **注意：** ダブルクォーテーション（ `"` ）で囲むのを忘れないでください。

1-6. 設定を確認する

```
git config --global user.name
```

自分の名前が表示されればOKです。

```
git config --global user.email
```

メールアドレスが表示されればOKです。

2. はじめてのリポジトリ（20分）

2-1. 作業フォルダを作る

デスクトップに作業用のフォルダを作ります。

```
cd ~/Desktop
```

```
mkdir my-profile
```

```
cd my-profile
```

> **解説：**
>
> - `cd` は「移動する」コマンドです
> - `mkdir` は「フォルダを作る」コマンドです
> - `~/Desktop` はデスクトップのことです

2-2. Gitリポジトリにする

今いるフォルダをGitで管理できるようにします。

```
git init
```

以下のような表示が出ればOKです。

```
Initialized empty Git repository in ...
```

2-3. 自己紹介ファイルを作る

テキストファイルを作成します。

```
notepad profile.txt
```

メモ帳が開きます。「ファイルが存在しません。新しく作成しますか？」と聞かれたら「はい」を押してください。

以下のように **自分の名前だけ** を書いて、保存して閉じてください。

```
名前：（自分の名前）
```

2-4. 状態を確認する

```
git status
```

赤い文字で `profile.txt` が表示されます。
これは「Gitがまだ追跡していないファイルがある」という意味です。

2-5. ファイルをGitに登録して、記録する

```
git add profile.txt
```

```
git status
```

今度は緑の文字で `profile.txt` が表示されます。「記録する準備ができた」という意味です。

```
git commit -m "名前を書いた"
```

> **解説：**
>
> - `git add` は「このファイルを記録対象にする」コマンドです
> - `git commit -m "..."` は「現在の状態を記録する」コマンドです
> - `"..."` の中はメッセージです。何を変更したか、短く書きます

おめでとうございます！ これが最初のコミットです。

コラム：コミットメッセージはなぜ大切なのか

今、 `git commit -m "名前を書いた"` と書きましたね。
この `"名前を書いた"` の部分が **コミットメッセージ** です。

コミットメッセージは「未来の自分や仲間への手紙」です。

たとえば、3ヶ月後にファイルの履歴を見返したとき、こんなメッセージが並んでいたらどう思いますか？

```
修正した
更新
fix
あ
```

何を変えたのか、さっぱりわかりません。
一方、こう書いてあったらどうでしょう？

```
名前を書いた
所属情報を追加した
趣味を追加した
意気込みの誤字を修正した
```

いつ、何が変わったのかが一目でわかります。

コミットメッセージは「自分のためのメモ」であると同時に、
チームメンバーが変更の経緯を理解するための記録 でもあります。

現場ではルールがある

実際の開発現場では、コミットメッセージの書き方にチームごとのルールが定められていることが多いです。

例えば：

```
feat: ユーザー登録機能を追加
fix: ログイン画面のエラーメッセージを修正
docs: README にセットアップ手順を追記
```

このように、変更の種類を先頭につけるルール（Conventional Commitsなど）を
採用しているチームもあります。

大切なのは、自分勝手に書くのではなく、チームのルールに従うことです。
配属されたら、まずチームのコミットメッセージのルールを確認しましょう。

Pull Request（PR）との関係

この研修の2日目で「Pull Request（PR）」という仕組みを体験します。
PRとは、自分の変更をチームに「確認してもらう」ための仕組みです。

チームメンバーがPRを確認するとき、まず見るのは **コミットメッセージの一覧** です。
メッセージがわかりやすければ、「何をやったのか」がすぐに伝わります。
メッセージがいい加減だと、コードの中身を全部読まないと意図が伝わりません。

つまり、**よいコミットメッセージは、チームの仕事をスムーズにする**のです。

> **今日のハンズオンでは：**
> まずは「何をしたかを短く日本語で書く」ことを意識してください。
> それだけで十分です。書き方の細かいルールは、配属先のチームで学びましょう。

3. 変更して、記録する、を繰り返す（40分）

ここからは `profile.txt` に内容を追記しては記録する、という作業を繰り返します。
毎回同じ流れ です。

```
（ファイルを編集・保存） → git status → git add → git commit
```

この流れを体で覚えてください。

3-1. 所属情報を追記する

```
notepad profile.txt
```

ファイルの続きに以下を追記して、保存して閉じます。

```
所属：（自分の部署や配属先）
入社年：2026年
```

状態を確認します。

```
git status
```

赤い文字で `modified: profile.txt` と出ます。「変更されたけど、まだ記録されていない」という意味です。

変更内容を確認してみましょう。

```
git diff
```

追加した行が `+` マーク付きで緑色に表示されます。これが「差分」です。

記録します。

```
git add profile.txt
```

```
git commit -m "所属情報を追加した"
```

3-2. 趣味を追記する

```
notepad profile.txt
```

続きに以下を追記して、保存して閉じます。

```
趣味：（自分の趣味）
```

記録します。

```
git status
```

```
git add profile.txt
```

```
git commit -m "趣味を追加した"
```

3-3. 好きな食べ物を追記する

```
notepad profile.txt
```

続きに追記して、保存して閉じます。

```
好きな食べ物：（好きな食べ物）
```

記録します。

```
git status
```

```
git add profile.txt
```

```
git commit -m "好きな食べ物を追加した"
```

3-4. 意気込みを追記する

```
notepad profile.txt
```

続きに追記して、保存して閉じます。

```
意気込み：（仕事への意気込みを自由に書いてください）
```

記録します。

```
git status
```

```
git add profile.txt
```

```
git commit -m "意気込みを追加した"
```

ここまでのprofile.txtの中身（例）

```
名前：山田太郎
所属：営業部
入社年：2026年
趣味：映画鑑賞
好きな食べ物：カレーライス
意気込み：早く仕事を覚えて、チームに貢献できるようになりたいです！
```

4. 履歴を振り返る（30分）

4-1. コミット履歴を見る

```
git log
```

5つのコミットが新しい順に表示されます。
それぞれに以下の情報がついています。

- コミットID（長い英数字）
- 作者（さっき設定した名前とメールアドレス）
- 日時
- メッセージ

> **操作：** `git log` の画面はスクロールできます。
>
> - `Enter` キー：1行ずつ進む
> - `Space` キー：1画面ずつ進む
> - `q` キー：終了する（**これを忘れると抜けられません！**）

4-2. 短い形式で見る

```
git log --oneline
```

1行ずつコンパクトに表示されます。こちらの方が見やすいです。

4-3. 最初と最新を比べる

最初のコミットと今の状態を比べてみましょう。

まず、最初のコミットのIDを確認します。

```
git log --oneline
```

一番下に表示されるのが最初のコミットです。その左側にある短い英数字（例： `a1b2c3d` ）をコピーしてください。

```
git diff コピーしたID HEAD
```

> **解説：** `HEAD` は「今の最新の状態」を意味します。

最初は名前だけだったファイルが、ここまで育ったことが差分として表示されます。
これが **バージョン管理** です。

5. GitHubにあげる（30分）

5-1. GitHubにリポジトリを作る（練習）

まずは **練習用** のリポジトリを作って、すぐに削除します。
「作って、消す」を先に体験しておきましょう。

> **なぜ削除を練習するのか？**
> 実際の業務では、会社の開発リポジトリに招待される形で参加します。
> その場合、リポジトリを削除する機会はまずありません。
> （むしろ、絶対に消してはいけないリポジトリばかりです。）
>
> 自分で自由に消せるリポジトリがあるのは **今だけ** です。
> クラウドサービスは「作ったら消せる」ことを知っておくと、怖さがなくなります。

リポジトリを作成する

1. ブラウザで https://github.com にアクセスし、ログインします
2. 右上の「+」ボタンから「New repository」を選びます
3. 以下のように設定します
   - Repository name: `test-repo`
   - Description: （空欄でOK）
   - **Private** を選択してください
   - 「Add a README file」に **チェックを入れてください**（今回だけ）
4. 「Create repository」をクリックします

リポジトリが作成され、READMEファイルが表示されるページが開きます。
おめでとうございます。これがあなたの最初のGitHubリポジトリです。

リポジトリを削除する

では、今作ったリポジトリを削除してみましょう。

1. リポジトリのページで、上部の「Settings」タブをクリックします
2. 一番下までスクロールします
3. 「Danger Zone」というセクションが見つかります
4. 「Delete this repository」をクリックします
5. 確認画面が表示されます。**リポジトリ名を入力する**よう求められます
6. `自分のユーザー名/test-repo` と正確に入力します
7. 「I understand the consequences, delete this repository」をクリックします

> **注目してほしいポイント：**
>
> - 削除は「Danger Zone（危険地帯）」の中にあります。
> GitHubは、間違って消さないよう何段階もの確認を挟んでいます。
> - リポジトリ名を手入力させるのも、誤操作を防ぐためです。
> コピー&ペーストではなく、自分の手でタイプすることに意味があります。

> クラウドサービスの「削除」は重い操作です。
> この慎重さを、今のうちに体で覚えておいてください。

リポジトリの一覧に戻って、 `test-repo` が消えていることを確認しましょう。

5-2. GitHubにリポジトリを作る（本番）

練習が終わったので、今度はハンズオンで使う **本番のリポジトリ** を作ります。

1. 右上の「+」ボタンから「New repository」を選びます
2. 以下のように設定します
   - Repository name: `my-profile`
   - Description: （空欄でOK）
   - **Private** を選択してください
   - 「Add a README file」のチェックは **外したまま** にしてください
3. 「Create repository」をクリックします

> **重要：GitHubにはPublicとPrivateの2種類があります。**
>
> - **Public：** インターネット上の誰でも見ることができます
> - **Private：** 自分だけが見ることができます（招待した人も見られます）
>
> 今回は個人情報を含むので **Private** を選びます。
> 仕事で使うリポジトリも基本的にPrivateです。

5-3. ローカルとGitHubを接続する

リポジトリを作成すると、画面に手順が表示されます。
「…or push an existing repository from the command line」の部分を使います。

Git Bashで以下を **1行ずつ** 実行します。

```
git remote add origin https://github.com/自分のユーザー名/my-profile.git
```

> **注意：** `自分のユーザー名` の部分は、GitHubのユーザー名に置き換えてください。
> GitHubの画面に表示されているURLをそのままコピーすると確実です。

```
git branch -M main
```

```
git push -u origin main
```

> **初回の `push` 時に、GitHubの認証を求められることがあります。**
> 画面の指示に従って、ブラウザでの認証を完了してください。
> 困ったら講師を呼んでください。

5-4. GitHubで確認する

ブラウザでGitHubのリポジトリページを開きます（またはリロードします）。

- `profile.txt` が表示されていますか？
- クリックして中身を見てみましょう。自分が書いた内容が表示されるはずです

自分のPCで書いたものが、インターネット上のGitHubに保存されました！

6. もうひとつファイルを追加する（10分）

6-1. README ファイルを作る

```
notepad readme.txt
```

以下の内容を書いて、保存して閉じます。

```
これは自己紹介リポジトリです。
Git研修のハンズオンで作成しました。
```

6-2. 記録してGitHubにあげる

```
git status
```

```
git add readme.txt
```

```
git commit -m "README を追加した"
```

```
git push
```

> **解説：** 2回目以降の `push` は `git push` だけでOKです。
> 最初の1回で接続先を覚えさせたので、行き先を指定する必要がありません。

ブラウザでGitHubを確認してみましょう。 `readme.txt` が追加されていますか？

7. Day 1のまとめ

今日使ったコマンド

コマンド	意味
`git config`	Gitの設定をする
`git init`	フォルダをGitリポジトリにする
`git status`	今の状態を確認する
`git add`	ファイルを記録対象にする
`git commit -m "..."`	変更を記録する
`git log`	記録の履歴を見る
`git diff`	変更の差分を見る
`git remote add`	GitHubと接続する
`git push`	GitHubに送る

今日の成果

- `profile.txt` を少しずつ書き上げて、6回以上コミットしました
- `readme.txt` を追加しました
- GitHubに自分のリポジトリを作り、公開しました
- `git status` → `git add` → `git commit` の流れが身についたはずです

明日は「ブランチ」と「Pull Request」に挑戦します。おつかれさまでした！